

Clase: Mónadas en TypeScript

Materia: Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI **Tema:** 05 | **Módulo:** II **Duración:** 120 minutos | **Fecha:** _____

Cómo usar esta minuta: Cada sección corresponde a una filmina. Los momentos (▶) son acciones secuenciales dentro de esa filmina. El texto entrecomillado es lo que decís en voz alta. El código está inline — no necesitás abrir `filminas.md` para dar la clase.

Objetivos de la Clase

- OA-1: Explicar qué problema resuelven las mónadas y por qué surgen como patrón.
 - OA-2: Identificar las tres leyes monádicas y verificarlas en código TypeScript.
 - OA-3: Implementar desde cero `Maybe`, `Either` e `IO` en TypeScript.
 - OA-4: Encadenar operaciones con `flatMap` / `bind` para pipelines funcionales.
 - OA-5: Contrastar la implementación en TS con la aproximación idiomática en Clojure.
 - OA-6: Reconocer mónadas en APIs existentes: `Promise`, `Array.flatMap`, `some->`.
-

BLOQUE 1 — Motivación: ¿por qué mónadas? (20 min)

[F-01] Portada

Tiempo: 1 min

▶ Al mostrar la portada

“Hoy vamos a construir la abstracción más poderosa de la programación funcional. La buena noticia: ya la usaron. `Promise.then` es una mónada. `Array.flatMap` es una mónada. `some->` en Clojure es una mónada. Hoy vamos a entender por qué.”

▶ Transición: “Empezamos con el problema que las mónadas resuelven.”

[F-02] El problema del encadenamiento

Tiempo: 3 min

▶ Al mostrar el concepto

“Imaginen que tienen tres funciones: buscar un usuario, obtener su dirección, obtener su código postal. Cada una puede fallar. ¿Cómo las encadenan?”

Conceptos clave para desarrollar:

- En el Tema 04 construimos `Result<T, E>` — era un `Either` simplificado. Pero solo manejábamos *una* operación. ¿Qué pasa con un pipeline de 3, 5, 10 operaciones que pueden fallar?
- El problema no es el error individual — es la **composición** de operaciones con efectos.
- Cada operación puede producir: un valor (`Just`), nada (`Nothing`), un error tipado (`Left`), o un efecto (`IO`). El código defensivo crece linealmente con cada paso.

► Pregunta a la clase:

“¿Cuántos `if/null` necesitarían si el pipeline tiene 5 pasos? ¿Y si tienen que propagar *por qué* falló cada uno?”

► Transición: “Veamos el código concreto.”

[F-03] Encadenamiento roto — TypeScript

Tiempo: 3 min

► Al mostrar el código

“Esto es lo que pasa cuando encadenamos tres operaciones que pueden devolver `null`.”

Código inline:

```
function getPostalCode(userId: number): string | null {  
  const user = findUser(userId);  
  if (user === null) return null;  
  
  const address = getAddress(user);  
  if (address === null) return null;  
  
  const postalCode = getPostalCode(address);  
  if (postalCode === null) return null;  
  
  return postalCode;  
}
```

Conceptos clave para desarrollar:

- Cada `if (x === null) return null` es un **cortocircuito manual**.
- Con 3 pasos tenemos 3 guardas. Con 10 pasos, 10 guardas. La lógica de dominio (buscar usuario, obtener dirección) queda enterrada entre guardas repetitivas.
- El patrón es siempre el mismo: “si el paso anterior produjo `null`, devolver `null`; si no, seguir”. Esto **grita** abstracción.

“¿Ven el patrón repetido? Ahora veamos el mismo problema en Clojure.”

► Transición: “Clojure tiene el mismo problema, con distinta sintaxis.”

[F-04] Encadenamiento roto — Clojure

Tiempo: 3 min

► Al mostrar el código

“En Clojure, `nil` es el equivalente de `null`. Y `when-let` es la guardia.”

Código inline:

```
(defn get-postal-code [user-id]
  (when-let [user (find-user user-id)]
    (when-let [address (get-address user)]
      (when-let [postal (get-postal-code address)]
        postal)))))
```

Conceptos clave para desarrollar:

- El anidamiento crece igual que en TypeScript. La indentación acumula niveles por cada operación.
- Peor aún: `when-let` con `nil` solo dice “no hay valor” — no dice *por qué* no hay valor. Si necesitamos propagación de errores, `when-let` no alcanza.
- Mismo patrón, dos lenguajes, misma frustración → necesitamos la misma solución abstracta.

► Transición: “Miremos esto como tabla.”

[F-05] Mismo problema, dos lenguajes

Tiempo: 2 min

► Al mostrar la tabla

“TypeScript y Clojure tienen el mismo problema estructural. Cambia la sintaxis, no el patrón.”

► Enfatizar:

- La columna “Razón del fallo: Perdida” es la clave. `null` / `nil` no explican nada.
- La fila “Solución necesaria” es donde entramos nosotros: una abstracción que encadene y propague contexto.

► Transición: “¿Recuerdan `map` ? Es el primer paso hacia la solución.”

[F-06] ¿Qué es una mónada? — la explicación simple

Tiempo: 5 min

► Al mostrar la filmina

“Antes de meternos en código, quiero que entiendan la idea con una analogía.”

► Explicar la analogía del sobre certificado:

“Imaginen un sobre certificado. Meter la carta es `of`. El cartero que abre el sobre, procesa el contenido y lo mete en un nuevo sobre certificado es `flatMap`. Ahora, la parte más importante: si el sobre llega vacío, el cartero no hace nada — pasa el sobre vacío al siguiente. Eso es el cortocircuito automático.”

Conceptos clave para desarrollar:

- Una mónada es un **patrón**, no una cosa misteriosa. Es un tipo que envuelve un valor y agrega un contexto: “puede no haber valor”, “puede haber un error”, “es un efecto pendiente”.
- El valor de adentro no se toca directamente — se usa `flatMap` para operar sobre él. Esto parece una restricción, pero es la fuente del poder: la mónada maneja el caso de fallo automáticamente.
- NO empezar desde la definición matemática. La analogía primero. El álgebra viene después, cuando ya entienden el patrón.

“En una frase: una mónada envuelve valores en un contexto y permite encadenar operaciones, manejando automáticamente los casos de fallo.”

► Transición: “Veamos cómo se ve esto en código real.”

[F-06b] Antes vs después — sin mónadas vs con mónadas

Tiempo: 3 min

► Al mostrar el código lado a lado

“A la izquierda, el código que ya escribieron: tres guardas `if/nullable` repetitivas. A la derecha, el mismo pipeline con `flatMap`: tres líneas, sin un solo `if`.”

Conceptos clave para desarrollar:

- Señalar que la lógica de “si falló, propagá el fallo” está repetida 3 veces en el código de la izquierda. En el de la derecha, está codificada *una sola vez* dentro de `flatMap`.
- No importa que no conozcan aún la implementación de `flatMap` — el punto es que **existe** una abstracción que elimina la repetición.
- Preguntar: “¿Cuántas guardas necesitarían si el pipeline tuviera 10 pasos?”

► Transición: “Ahora entendamos *por qué* necesitamos flatMap y no alcanza con map.”

[F-06c] De `map` a `flatMap` — la diferencia técnica

Tiempo: 3 min

► Al mostrar el diagrama

“Esta es la diferencia técnica clave.”

Conceptos clave para desarrollar:

- `map` transforma el valor de adentro y envuelve el resultado en una caja. Si la función *ya* devuelve una caja, quedan dos cajas anidadas: `Maybe<Maybe<T>>`. Eso no sirve.
- `flatMap` aplica la función y no re-envuelve porque la función ya devuelve una caja. Resultado: una sola caja.
- La diferencia en código es una línea: `just(f(m.value))` vs `f(m.value) . map` agrega `just(...)`, `flatMap` no.
- Cerrar con la definición de trabajo: `of` + `flatMap` = mónada.

► En la pizarra:

```
map:      Maybe<User> → (User → Maybe<Address>) → Maybe<Maybe<Address>> ❌  
flatMap: Maybe<User> → (User → Maybe<Address>) → Maybe<Address>      ✅
```

► Pregunta:

“¿Por qué `map` genera doble envoltorio y `flatMap` no?”

► Transición: “Una analogía visual más para fijar, y después construimos la primera mónada.”

[F-07] Analogía del contenedor

Tiempo: 3 min

► Al mostrar el diagrama

“Piensen en una mónada como una caja con reglas.”

Conceptos clave para desarrollar:

- `of(valor)` : mete el valor en la caja. La caja puede ser `Maybe` (caja que puede estar vacía), `Either` (caja que puede tener un error), `IO` (caja que difiere la ejecución).
- `flatMap(f)` : abre la caja, aplica `f` al valor de adentro. `f` devuelve una nueva caja. La mónada se encarga de que no queden cajas anidadas.
- Si la caja está vacía (`Nothing`) o tiene error (`Left`), `flatMap` *no abre la caja* — propaga el estado tal cual. Esa es la magia: el cortocircuito es automático.
- Usar la tabla de las tres cajas para anticipar lo que viene: `Maybe`, `Either`, `IO` — tres contextos distintos, mismo patrón de operación.

“Es como una cadena de montaje: cada estación recibe la caja, hace su trabajo, y la pasa. Si alguna estación señala «defectuoso», el producto pasa directo al final sin que las estaciones siguientes lo toquen.”

► Transición: “Ahora construimos la primera mónada: Maybe.”

BLOQUE 2 — Maybe: la mónada de la opcionalidad (25 min)

[F-08] Maybe en TypeScript — Definición

Tiempo: 3 min

► Al mostrar el tipo

“Esta es la definición más simple posible de Maybe en TypeScript.”

Código inline:

```
type Maybe<T> =  
  | { tag: 'Just'; value: T }  
  | { tag: 'Nothing' };  
  
const just = <T>(value: T): Maybe<T> =>  
  ({ tag: 'Just', value });  
  
const nothing = <T>(): Maybe<T> =>  
  ({ tag: 'Nothing' });
```

Conceptos clave para desarrollar:

- Tagged union discriminada por `tag` — TypeScript sabe que si `tag === 'Just'`, entonces `.value` existe. Si `tag === 'Nothing'`, `.value` no existe. Seguridad en compilación.
- Es el mismo patrón que `Result<T, E>` del Tema 04, pero más simple: no hay tipo de error, solo presencia o ausencia.
- `just` y `nothing` son constructores — funciones puras que crean valores Maybe.

► Transición: Las operaciones.

[F-09] Maybe en TypeScript — `of`, `map`, `flatMap`

Tiempo: 3 min

► Al mostrar las funciones

“Tres operaciones: `of` para envolver, `map` para transformar, `flatMap` para encadenar.”

Código inline:

```
const of = <T>(value: T): Maybe<T> => just(value);

const map = <T, U>(m: Maybe<T>, f: (x: T) => U): Maybe<U> =>
  m.tag === 'Just' ? just(f(m.value)) : nothing();

const flatMap = <T, U>(m: Maybe<T>, f: (x: T) => Maybe<U>): Maybe<U> =>
  m.tag === 'Just' ? f(m.value) : nothing();
```

Conceptos clave para desarrollar:

- `map` envuelve el resultado de `f` en `Just` → por eso puede generar `Maybe<Maybe<T>>` si `f` ya devuelve un `Maybe`.
- `flatMap` **no envuelve** — delega en `f`, que ya devuelve `Maybe<U>`. Por eso aplana.
- La diferencia en el código es una línea: `just(f(m.value))` vs `f(m.value)`. Eso es todo lo que distingue `map` de `flatMap`.
- Sobre `Nothing`: ambas operaciones devuelven `nothing()` sin ejecutar `f`. Eso es el cortocircuito automático.

► Transición: “Veamos un pipeline real.”

[F-10] Maybe en TypeScript — Pipeline completo

Tiempo: 4 min

► Al mostrar el pipeline

“Ahora el mismo pipeline de buscar usuario → dirección → código postal, pero sin `if/null`.”

Código inline:

```
const postalCode: Maybe<string> =
  flatMap(
    flatMap(findUser(1), getAddress),
    getPostal
  );
```

Conceptos clave para desarrollar:

- Comparen con la versión original de 12 líneas con 3 `if`. Acá son 4 líneas, sin guardas.
- Si `findUser` devuelve `nothing()`, los dos `flatMap` subsiguientes propagan `nothing()` sin ejecutar nada. Es el mismo cortocircuito, pero codificado *una sola vez* dentro de `flatMap`.
- El pipeline lee de adentro hacia afuera (o con `pipe`: de arriba hacia abajo). Puede parecer menos intuitivo, pero es componible: agregar un paso más es agregar un `flatMap` más, no un `if` más.

► En el REPL (TypeScript): Ejecutar con `findUser` que devuelva `just({...})` y con una que devuelva `nothing()` para mostrar ambos caminos.

► Transición: “Ahora el mismo problema en Clojure.”

[F-11] Maybe en Clojure — idiomático con `some->`

Tiempo: 4 min

► Al mostrar el código

“Clojure ya tiene una solución nativa para este problema: el threading macro `some->` que corta en `nil`.”

Código inline:

```
(some-> 1
      find-user
      get-address
      get-postal)
;; => "9410" o nil
```

Conceptos clave para desarrollar:

- `some->` pasa el resultado de cada expresión a la siguiente. Si cualquier paso devuelve `nil`, el macro corta y devuelve `nil`.
- Es un Maybe *implícito*: no hay `Just / Nothing`, no hay wrapper. El `nil` de Clojure es el Nothing, y el threading macro es el `flatMap`.
- Ventaja: cero ceremonia. No hay que definir tipos, no hay que importar librerías.
- Limitación: solo funciona con `nil`. No puede distinguir entre “no encontrado” y “error en la base de datos”. Para eso necesitamos `Either`.

“En Clojure, `some->` es Maybe gratuitamente. Pero es un Maybe sin tipo y sin razón de fallo.”

► Transición: “¿Y si queremos Maybe explícito en Clojure?”

[F-12] Maybe en Clojure — explícito con `cats`

Tiempo: 4 min

► Al mostrar el código

“Si queremos formalidad — `mlet`, `bind`, leyes verificables — Clojure tiene la librería `cats`.”

Código inline:

```
(require '[cats.monad.maybe :as m])
(require '[cats.core :as mc])

;; mlet = do-notation para mónadas
(mc/mlet [user (find-user-m 1)
          address (get-address-m user)
          postal (get-postal-m address)]
  (mc/return postal))
```

Conceptos clave para desarrollar:

- `mlet` es syntactic sugar sobre `bind` encadenado. Cada línea puede fallar con `(m/nothing)` y todo el bloque cortocircuita.
- Es equivalente al `flatMap` anidado de TypeScript, pero con una sintaxis plana que se lee de arriba a abajo.
- En la práctica: `cats/maybe` se usa poco en producción Clojure. Los equipos prefieren `some->` por pragmatismo. Pero para enseñar el patrón y verificar las leyes, es perfecto.

► Transición: “Entonces, ¿cuándo usar cuál?”

[F-13] Maybe — `some->` vs `cats/maybe`

Tiempo: 2 min

► Al mostrar la comparación

“El criterio es simple: `some->` para producción, `cats/maybe` para entender el patrón.”

► Enfatizar:

- No es que uno sea “mejor” — son herramientas para contextos distintos.
- `some->` es *idiomático* en Clojure. `cats/maybe` es *didáctico*.
- En TypeScript no existe esta tensión: el tipo `Maybe<T>` es idiomático porque el sistema de tipos lo recompensa.

► Transición: “Resumamos Maybe en una tabla.”

[F-14] Comparativa Maybe — TS vs Clojure

Tiempo: 3 min

► Al mostrar la tabla

“Mismo concepto, dos expresiones muy distintas.”

Conceptos clave para desarrollar:

- **Type safety:** la diferencia fundamental. En TS, el compilador te dice si olvidaste manejar `Nothing`. En Clojure, un `nil` inesperado explota en runtime.
- **Ergonomía:** `some->` gana en brevedad por mucho. Pero brevedad ≠ seguridad.
- **Idiomático:** en TS, usar Maybe/Option (fp-ts, Effect) es práctica industrial cada vez más común. En Clojure, `cats` sigue siendo nicho.

► Pregunta:

“¿Cuándo elegirían la concisión de `some->` sobre la seguridad de `Maybe<T>`?”

► **Transición:** “Maybe maneja ausencia. ¿Qué pasa cuando necesitamos saber *por qué* algo falló? Ahí entra Either.”

BLOQUE 3 — Either: la mónada del error tipado (25 min)

[F-15] Either en TypeScript — Definición

Tiempo: 3 min

► Al mostrar el tipo

“Si `Maybe` es presencia/ausencia, `Either` es éxito/error con información del fallo.”

Código inline:

```
type Either<E, T> =  
  | { tag: 'Left'; error: E }  
  | { tag: 'Right'; value: T };
```

Conceptos clave para desarrollar:

- `Right` = camino feliz (right = correcto). `Left` = error con dato tipado.
- Convención universal: “right is right”. En Haskell, fp-ts, Effect, cats — todos usan esta convención.
- Comparado con Maybe: `Nothing` no dice nada. `Left({ field: 'email', msg: 'Inválido' })` dice *exactamente* qué falló y dónde.
- Comparado con `Result<T, E>` del Tema 04: es el **mismo concepto** con nombre estándar en la teoría de categorías.

► **Transición:** “Las operaciones.”

[F-16] Either en TypeScript — Operaciones

Tiempo: 2 min

► Al mostrar las funciones

“Las operaciones son idénticas en estructura a Maybe, con la diferencia de que Left propaga el error sin ejecutar la función.”

Código inline:

```
const flatMap = <E, T, U>(  
  m: Either<E, T>, f: (x: T) => Either<E, U>  
) => Either<E, U> =>  
  m.tag === 'Right' ? f(m.value) : m;
```

► Enfatizar:

- `m.tag === 'Right' ? f(m.value) : m` — si es `Right`, aplica `f`. Si es `Left`, **devuelve el mismo `Left`** (con su error intacto).
- El tipo `E` se preserva: el error original se propaga sin modificar a través de toda la cadena.

► Transición: “Veamos un caso de uso real.”

[F-17] Either en TypeScript — Validador de formulario

Tiempo: 5 min

► Al mostrar el pipeline completo

“Un validador de formulario con 3 campos. Cada validación puede fallar con un error tipado.”

Código inline:

```
type ValidationError = { field: string; message: string };

const validateName = (name: string): Either<ValidationError, string> =>
  name.length >= 2
    ? right(name)
    : left({ field: 'name', message: 'Mínimo 2 caracteres' });

const validateForm = (name: string, email: string, age: number) =>
  flatMap(validateName(name), validName =>
    flatMap(validateEmail(email), validEmail =>
      map(validateAge(age), validAge =>
        ({ name: validName, email: validEmail, age: validAge })
      )
    )
  );
```

Conceptos clave para desarrollar:

- Cada `validateX` devuelve `Either<ValidationError, T>`. Si falla, `left` con el error tipado.
- El pipeline con `flatMap` cortocircuita en la **primera** validación que falle. No ejecuta las siguientes.
- El error propagado contiene `field` y `message` — sabemos exactamente qué falló.
- Comparar con `try/catch`: con excepciones, el tipo del error es `unknown`. Con `Either`, es `ValidationError` — el compilador verifica.

► En el REPL: Ejecutar con datos válidos e inválidos. Mostrar cómo el error se propaga con su información intacta.

► Transición: “Comparemos con el enfoque clásico.”

[F-18] Either vs try/catch

Tiempo: 3 min

► Al mostrar la tabla

“Dos filosofías para el manejo de errores. Ninguna es siempre mejor — depende del contexto.”

Conceptos clave para desarrollar:

- **Flujo implícito** (`try/catch`): el error salta stack frames hasta encontrar un `catch` . Si nadie lo atrapa, crash. El programador no ve el error en la firma de la función.
- **Flujo explícito** (`Either`): el error está en el tipo de retorno. Cualquiera que llame a la función sabe que puede fallar. El compilador no te deja ignorar el caso de error.
- **Criterio pragmático**: errores verdaderamente excepcionales (disco lleno, out of memory) → excepción. Errores del dominio (validación, no encontrado) → `Either`.
- En la industria: `Effect` (TypeScript) y `Rust` (con `Result`) llevan esta filosofía al mainstream.

► Transición: “Ahora, `Either` en Clojure.”

[F-19] `Either` en Clojure — con `cats`

Tiempo: 4 min

► Al mostrar el código

“Mismo patrón de validación, ahora con la librería `cats` en Clojure.”

Código inline:

```
(mc/mlet [name (validate-name "Ana")
          email (validate-email "ana@mail.com")]
  (mc/return {:name name :email email}))
;; => #<Right {:name "Ana", :email "ana@mail.com"}>
```

Conceptos clave para desarrollar:

- `mlet` funciona idéntico al `flatMap` anidado de TypeScript, pero con sintaxis plana.
- `e/right` y `e/left` son los constructores — equivalentes a `right()` y `left()` en TS.
- Si `validate-name` devuelve `(e/left {...})` , la segunda línea no se ejecuta. Mismo cortocircuito.
- El resultado es un `Either` de `cats` — se puede inspeccionar con `(e/right? resultado)` o pattern matching.

► Transición: “Pero en Clojure hay otra forma de hacer esto...”

[F-20] `Either` en Clojure — idiomático sin `cats`

Tiempo: 4 min

► Al mostrar el código

“La forma más común en Clojure de producción: mapas con keywords convencionales.”

Código inline:

```
(defn validate-name [name]
  (if (>= (count name) 2)
    {:ok name}
    {:error {:field :name :msg "Mínimo 2 chars"}}))

(defn validate-form [name email]
  (let [r1 (validate-name name)]
    (if (:error r1)
      r1
      (let [r2 (validate-email email)]
        (if (:error r2)
          r2
          {:ok {:name (:ok r1) :email (:ok r2)}}))))))
```

Conceptos clave para desarrollar:

- Funciona. Es Clojure idiomático: datos simples (mapas), sin librería, sin tipos especiales.
- **Pero:** el anidamiento vuelve a crecer. No hay `flatMap` genérico — el encadenamiento es manual.
- Es un Either reimplementado ad-hoc, con menos ergonomía y sin garantías formales.
- Riesgo real: sin convención estricta, un equipo usa `:ok/:error`, otro usa `:result/:failure`. Bugs silenciosos.

“Es la tensión central de Clojure: datos simples y flexibles, pero sin la red de seguridad de los tipos.”

► **Transición:** “Esta tensión merece una filmina propia.”

[F-21] Clojure: ¿datos simples o mónadas formales?

Tiempo: 3 min

► Al mostrar el concepto

“Esta es una de las discusiones más interesantes de la materia.”

Conceptos clave para desarrollar:

- **Filosofía Clojure:** “data is the universal interface” (Rich Hickey). Un mapa es más flexible que un tipo: se puede serializar, inspeccionar en el REPL, extender sin recompilar.
- **Precio de la informalidad:** sin flatMap genérico, sin exhaustividad del compilador, y con convenciones que dependen de la disciplina del equipo.
- **TypeScript:** el sistema de tipos *recompensa* la formalidad. Autocompletado, errores en compilación, refactoring seguro.
- **Criterio:** no es que uno sea “mejor” — son compromisos distintos. TS cambia flexibilidad por seguridad. Clojure cambia seguridad por simplicidad.

► **Transición:** “Tabla comparativa final de Either.”

[F-22] Comparativa Either — TS vs Clojure

Tiempo: 2 min

► Al mostrar la tabla

“Either es donde la diferencia entre tipado estático y dinámico se siente más fuerte.”

► Enfatizar:

- La fila “Exhaustividad” es clave: en TS, olvidar manejar `Left` es un error de compilación. En Clojure, es un bug de producción.
- La fila “Idiomático”: en TS, fp-ts/Effect hacen que Either sea mainstream. En Clojure, `cats/either` sigue siendo nicho porque los mapas son “suficientemente buenos” para la mayoría.

► Transición: “Ya construimos Maybe y Either. Falta IO: la mónada de los efectos.”

BLOQUE 4 — IO, leyes monádicas y mónadas ocultas (25 min)

[F-23] IO en TypeScript — Definición

Tiempo: 3 min

► Al mostrar el código

“IO es la mónada más radical: convierte un efecto (leer archivo, mostrar en pantalla) en un valor puro.”

Código inline:

```
type IO<T> = { run: () => T };

const ioOf = <T>(value: T): IO<T> =>
  ({ run: () => value });

const ioFlatMap = <T, U>(io: IO<T>, f: (x: T) => IO<U>): IO<U> =>
  ({ run: () => f(io.run()).run() });
```

Conceptos clave para desarrollar:

- `IO<T>` no contiene el valor — contiene la *receta* para obtenerlo. Es un thunk tipado.
- Nada se ejecuta al crear el IO. Solo se ejecuta al llamar `.run()`.
- Esto permite componer efectos sin ejecutarlos: el pipeline es una descripción, no una ejecución.
- Analogía: un IO es como un guión de teatro. Escribirlo no es actuar. `.run()` es subir al escenario.

► Transición: “Veamos un ejemplo concreto.”

[F-24] IO en TypeScript — Ejemplo

Tiempo: 3 min

► Al mostrar el pipeline

“Leemos un nombre, saludamos. Nada se ejecuta hasta `.run()`.”

Código inline:

```
const readLine: IO<string> =
  ({ run: () => prompt("Ingrese su nombre:") ?? "" });

const greet = (name: string): IO<string> =>
  ({ run: () => {
    const msg = `Hola, ${name.toUpperCase()}!`;
    console.log(msg);
    return msg;
  }});

const program: IO<string> = ioFlatMap(readLine, greet);
// Hasta acá: NADA se ejecutó. 'program' es una descripción pura.

program.run(); // Ahora sí: prompt + console.log
```

Conceptos clave para desarrollar:

- La línea `const program = ioFlatMap(readLine, greet)` no tiene efectos. Es composición pura.
- Solo `program.run()` dispara la cadena de efectos.
- Esto es lo que hace Haskell con todo el I/O. TypeScript no lo obliga, pero podemos elegirlo para partes críticas del sistema.

► Transición: “¿Clojure necesita IO?”

[F-25] IO en Clojure — thunks y `delay`

Tiempo: 3 min

► Al mostrar el código

“Respuesta corta: no. Clojure es impuro por defecto, y esa es una decisión de diseño deliberada.”

Código inline:

```
;; Efecto directo – idiomático
(println "Hola") ; se ejecuta inmediatamente

;; Diferir con delay
(def greeting (delay (str "Hola, " (read-line))))
@greeting ; force: ejecuta y cachea

;; core.async: composición de efectos async
(go (>! c (str "Hola, " (<! (go (read-line))))))
```

Conceptos clave para desarrollar:

- Clojure *eligió* no requerir IO wrappers. La impureza es explícita en la documentación y la convención, no en el tipo.
- `delay` permite diferir, pero no compone como IO. Es un thunk de un solo uso, no una mónada.
- `core.async / go` blocks son lo más cercano a composición de efectos en Clojure: canales como piping de valores asíncronos.
- Contraste: TS necesita IO para ser “puro hasta el borde”; Clojure elige pragmatismo y controla efectos por convención.

► Transición: “Tabla comparativa.”

[F-26] Comparativa IO — TS vs Clojure

Tiempo: 2 min

► Al mostrar la tabla

“IO es donde la diferencia filosófica entre los lenguajes es más clara.”

► Enfatizar que:

- En TS, IO es una **elección** que el sistema de tipos recompensa.
- En Clojure, IO formal **no existe** porque el lenguaje no la necesita — la cultura del lenguaje ya distingue lógica pura de efectos por convención.

► Transición: “Ahora las reglas del juego: las tres leyes monádicas.”

[F-27] Las tres leyes monádicas

Tiempo: 3 min

► Al mostrar las leyes

“Toda mónada debe cumplir tres leyes. No son opcionales: si no las cumple, el flatMap puede comportarse de forma inesperada.”

Conceptos clave para desarrollar:

- **Identidad izquierda:** `of(a).flatMap(f) === f(a)`. Envolver un valor y aplicar una función es lo mismo que aplicar la función directamente. `of` no agrega comportamiento.
- **Identidad derecha:** `m.flatMap(of) === m`. Si aplicás `flatMap` con la función que solo envuelve, recuperarás el original. `of` es neutra.
- **Asociatividad:** `m.flatMap(f).flatMap(g) === m.flatMap(x => f(x).flatMap(g))`. El orden de agrupación de los `flatMap` no importa. Esto permite refactorizar pipelines con confianza.
- Analogía: las leyes son como las leyes de la aritmética (asociatividad de la suma, elemento neutro). No las verificamos cada vez que sumamos, pero sin ellas la aritmética no funcionaría.

► Transición: “Verifiquemos con código.”

[F-28] Leyes verificadas en TypeScript

Tiempo: 3 min

► Al mostrar los tests

“Cada ley se verifica comparando ambos lados de la ecuación.”

► En el REPL: Ejecutar los tres `console.assert` en vivo. Mostrar que ninguno falla.

Conceptos clave para desarrollar:

- Usamos `JSON.stringify` para comparar estructuralmente (no por referencia). En producción usaríamos una función `equals`.
- Si una implementación de Maybe no cumple la ley 3, significa que refactorizar el orden de `flatMap` podría cambiar el resultado — pipeline poco confiable.

► Transición: “Lo mismo en Clojure.”

[F-29] Leyes verificadas en Clojure

Tiempo: 3 min

► Al mostrar el REPL

“En Clojure, las leyes se verifican en el REPL con `assert`.”

► Ejecutar en REPL en vivo. Las tres leyes pasan con `cats/maybe`.

► Enfatizar:

- `some->` no tiene leyes formales porque no es una mónada — es un macro. No podemos verificar asociatividad de un macro.
- `cats/maybe` sí cumple las leyes porque está modelada como mónada formal.
- Esto refuerza la distinción: `some->` es práctico, `cats/maybe` es teóricamente correcto.

► Transición: “Ahora, las mónadas que ya conocían sin saberlo.”

[F-30] Mónadas escondidas — `Promise`

Tiempo: 3 min

► Al mostrar el código

“Promise es la mónada más usada en JavaScript/TypeScript. Solo que nadie la llama así.”

Conceptos clave para desarrollar:

- `Promise.resolve(42)` = `of(42)` . Envuelve un valor en el contexto de asincronía.
- `.then(f)` = `flatMap(f)` cuando `f` devuelve una Promise. JavaScript aplanar automáticamente `Promise<Promise<T>>` a `Promise<T>` .
- ¿Por qué “casi” mónada? Promise es *eager*: se empieza a ejecutar al crearla. IO es *lazy*: no se ejecuta hasta `.run()` . Promise viola la pureza pero cumple (casi) las leyes.
- Promise no cumple *estrictamente* la ley de identidad izquierda por el aplanamiento automático en casos límite con `then` , pero en la práctica funciona como mónada.

► Pregunta:

“Si `Promise.then` es `flatMap` , ¿qué efecto modela Promise?”

► Transición: “Hay más mónadas ocultas.”

[F-31] Mónadas escondidas — más ejemplos

Tiempo: 2 min

► Al mostrar la tabla

“Cada fila de esta tabla es un patrón monádico que ya estaban usando.”

► Recorrer la tabla fila por fila:

- `Array.flatMap` : “si tenemos `[1,2,3]` y una función que devuelve arrays, `flatMap` aplanar el resultado. Es la List monad — no-determinismo.”
- `?.` optional chaining: “es Maybe sin el tipo. `user?.address?.postalCode` corta en undefined.”
- `some->` en Clojure: “ya lo vimos — Maybe implícito.”
- `for` en Clojure: “comprehension con bindings — es la List monad con syntactic sugar.”
- `go` blocks: “composición de operaciones asíncronas por canales. IO/async.”

► Transición: “Veamos dónde encaja todo esto en la jerarquía formal.”

[F-32] Jerarquía: Functor → Monad

Tiempo: 3 min

► Al mostrar el diagrama

“Functor, Applicative, Monad: tres niveles de poder, cada uno agrega una capacidad.”

Conceptos clave para desarrollar:

- **Functor** (`map`): puede transformar el valor dentro del contexto. `Maybe.map` , `Array.map` , `Promise.then(f)` cuando `f` no devuelve contexto.
- **Applicative** (`ap`): puede aplicar una *función* envuelta a un *valor* envuelto. Útil para validaciones paralelas (todas a la vez, no cortocircuito secuencial). No profundizamos — mención.
- **Monad** (`flatMap` / `bind`): puede encadenar operaciones que producen contexto. Es el nivel más poderoso y el que más usamos.
- Haskell inventó la terminología. No necesitamos saber Haskell, pero entender que `>>=` es `flatMap` conecta toda la literatura.

“No necesitan memorizar la jerarquía. Solo sepan que existe, y que Monad es la que más importa en la práctica.”

► **Transición:** “Ahora, el ecosistema industrial.”

BLOQUE 5 — Ecosistemas, IA y reflexión final (15 min)

[F-33] fp-ts y Effect — ecosistema TS

Tiempo: 3 min

► **Al mostrar el código**

“No tenemos que implementar Maybe y Either a mano en producción. Estas librerías ya lo hicieron.”

Conceptos clave para desarrollar:

- **fp-ts**: desde 2019. Basada en la tradición Haskell. `pipe` , `O.fromNullable` , `E.flatMap` . Es la más madura.
- **Effect**: desde 2023. Enfoque moderno con do-notation (`Effect.gen`), built-in concurrencia, tipado de errores y dependencias. Es la dirección de la industria.
- Ambas implementan las tres mónadas que construimos (Option, Either, IO/Effect) con tests de leyes, funciones auxiliares, y documentación completa.
- Mensaje: “lo que construimos a mano en 10 líneas, estas librerías lo industrializan con cientos de funciones auxiliares.”

► **Transición:** “¿Y en Clojure?”

[F-34] Ecosistema Clojure — cats y más

Tiempo: 2 min

► **Al mostrar el código**

“cats es la librería estándar de mónadas para Clojure, pero su adopción es menor que fp-ts.”

► **Enfatizar:**

- `cats` (funcool): Maybe, Either, State, Writer, Reader — similar a fp-ts.
- `manifold` : promesas composicionales. `d/chain` es un `flatMap` sobre `deferred`. Usado en producción (servidor Aleph).
- Adopción: menor que en TS. La comunidad Clojure prefiere datos simples + convenciones. Rich Hickey ha dicho explícitamente que las mónadas formales no encajan en la filosofía del lenguaje.

► **Transición:** “Conectemos esto con IA.”

[F-35] Pipeline IA con mónadas

Tiempo: 3 min

► **Al mostrar el diagrama**

“El pipeline de un agente IA es exactamente el mismo patrón que el validador de formulario.”

Conceptos clave para desarrollar:

- Pipeline: `prompt → LLM → parseo → validación → respuesta`. Cada paso puede fallar: prompt vacío, timeout del modelo, JSON malformado, respuesta inválida.
- En TypeScript: cada paso devuelve `Either<Error, T>`. `flatMap` encadena y cortocircuita en el primer fallo.
- En Clojure: `go` blocks con mapas `{:ok/:error}` — misma lógica, distinta sintaxis.
- Conexión didáctica: “el `flatMap` que usaron para validar un formulario es el mismo que compone un pipeline de IA. Es el patrón más general que existe para composición con efectos.”

► **Transición:** “Cerramos con la reflexión más importante.”

[F-36] Reflexión: ¿mónadas formales en Clojure?

Tiempo: 4 min

► **Gestionar como discusión abierta**

“Voy a lanzar tres preguntas. Espero argumentos de las dos orillas.”

► **Moderar la discusión:**

1. "Si el compilador no te obliga a usar mónadas, ¿por qué hacerlo?" — Esperar respuestas. Guiar hacia: disciplina, comunicación de intenciones, testability.
2. " `some->` y mapas `{:ok/:error}` ¿son suficiente mónada para Clojure?" — Esperar. Guiar hacia: depende del tamaño del equipo y la criticidad. Equipo chico que se conoce: sí. Equipo grande con rotación: probablemente no.
3. "¿En qué lenguaje es más natural usar mónadas explícitas?" — Consenso: TS, porque el sistema de tipos las recompensa. Pero el *concepto* es universal.

"La conclusión provisoria: el concepto es universal, la forma de expresarlo depende del lenguaje. Usá mónadas cuando el error o efecto es parte explícita del dominio. No para sumar dos números."

► Transición: "Sinteticemos todo."

[F-37] Síntesis: 3 mónadas × 2 lenguajes

Tiempo: 2 min

► Al mostrar la tabla

"Todo lo que vimos en una sola tabla."

► Recorrer las tres filas señalando el patrón común (`of` + `flatMap` + leyes) y la diferencia esencial (tipos vs convenciones).

► Transición: "Guía de decisión."

[F-38] ¿Cuándo usar cuál?

Tiempo: 2 min

► Al mostrar la tabla

"No toda función necesita una mónada. Acá está el criterio."

► Recorrer las filas breve:

- Valor puede no existir → Maybe (no `null`).
- Operación falla con info → Either (no `try/catch` para errores de dominio).
- Efecto lateral → IO (separar descripción de ejecución).
- Async → Promise/core.async (ya lo usan).

► Transición: "Cerramos."

CIERRE (10 min)

[F-39] Recapitulación visual

Tiempo: 4 min

► Al mostrar el diagrama

“Los 7 puntos que se llevan de esta clase.”

► Repasar los 7 puntos del diagrama uno por uno, breve:

1. Problema → encadenamiento con efectos.
 2. Solución → `of` + `flatMap` = mónada. 3-5. Maybe, Either, IO en TS y Clojure.
 3. Leyes — contrato de corrección.
 4. El concepto es universal.
-

[F-40] Preguntas clave

Tiempo: 3 min

► Lanzar las 5 preguntas:

1. “¿Cuándo usar Maybe vs Either?” → Maybe: solo ausencia. Either: ausencia + razón.
 2. “¿Por qué flatMap y no solo map?” → map anida, flatMap aplana.
 3. “¿Clojure necesita mónadas formales?” → No las necesita, pero los patrones están por todas partes.
 4. “¿Promise.then cumple las tres leyes?” → Casi — eager evaluation rompe pureza pero no las leyes en práctica.
 5. “¿Cuál ley garantiza refactoring?” → Asociatividad (ley 3).
-

[F-41] Adelanto Tema 06

Tiempo: 2 min

► Al mostrar la preview

“La próxima clase: funcional en Python y su ecosistema IA. Van a ver cómo Python toma ideas de los dos mundos — tipos opcionales como TypeScript, datos simples como Clojure.”

[F-42] Cierre

Tiempo: 1 min

► Al mostrar la filmina de cierre

"Promise.then, Array.flatMap, some->, optional chaining — ya las conocían. Ahora saben por qué funcionan. Eso es lo que separa usar una herramienta de entender un principio."

Material para el alumno

- [guia-estudio.md](#): Desarrollo completo de las 3 mónadas con implementaciones, leyes y ejercicios.
- **Código fuente**: implementaciones en TypeScript y Clojure disponibles en el repositorio de la materia.
- **Lecturas complementarias**: Wadler (1995), Anderlind & Åsberg (2023).